HW8-Packer & Terraform

Installation

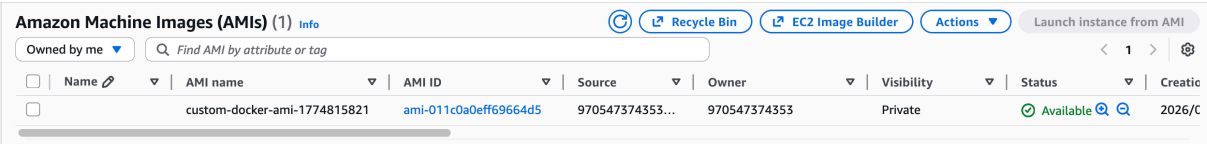
```
yj@YJs-MacBook-Air DevOps % brew install hashicorp/tap/terraform
==@ Fetching downloads for: terraform
✔ Formula terraform (1.14.8)                                          Verified     28.7MB/ 28.7MB
==6 Installing terraform from hashicorp/tap
🍺  /opt/homebrew/Cellar/terraform/1.14.8: 5 files, 95.7MB, built in 1 second
==> Running `brew cleanup terraform`...
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
yj@YJs-MacBook-Air DevOps % brew install hashicorp/tap/packer
==@ Fetching downloads for: packer
✔ Formula packer (1.15.1)                                             Verified     23.9MB/ 23.9MB
==6 Installing packer from hashicorp/tap
🍺  /opt/homebrew/Cellar/packer/1.15.1: 5 files, 77.9MB, built in 1 second
==> Running `brew cleanup packer`...
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
yj@YJs-MacBook-Air DevOps % aws --version && terraform --version && packer --version
aws-cli/2.28.1 Python/3.13.5 Darwin/25.3.0 source/arm64
Terraform v1.14.8
on darwin_arm64
Packer v1.15.1
yj@YJs-MacBook-Air DevOps %
```

ami.pkr.hcl

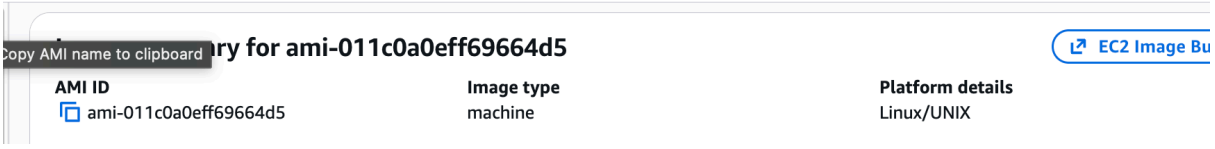
```
# aws ebs builder, launch temp EC2 instance from official Amazon Linux AMI
# plugin for amazon
packer {
  required_plugins {
    amazon = {
      source  = "github.com/hashicorp/amazon"
      version = "~> 1"
    }
  }
}

# ssh public key to bake into the AMI, directly look into local files
variable "ssh_public_key" {
  type    = string
  default = "~/.ssh/id_ed25519.pub"
}

# base image and instance config
source "amazon-ebs" "amazon-linux" {
  ami_name      = "custom-docker-ami-{{timestamp}}"
  instance_type = "t2.micro"
  region        = "us-west-2"
  ssh_username  = "ec2-user"

  # latest amazon linux 2023
  source_ami_filter {
    filters = {
      name                = "al2023-ami-*-x86_64"
      root-device-type    = "ebs"
      virtualization-type = "hvm"
    }
    most_recent = true
    owners      = ["amazon"]
  }
}

# install docker and inject ssh key
build {
  sources = ["source.amazon-ebs.amazon-linux"]

  provisioner "shell" {
    inline = [
      "sudo dnf update -y",
      "sudo dnf install -y docker",
      "sudo systemctl enable docker",
      "sudo systemctl start docker",
      "sudo usermod -aG docker ec2-user"
    ]
  }

  provisioner "shell" {
    inline = [
      "echo '${file(pathexpand(var.ssh_public_key))}' >> /home/ec2-user/.ssh/authorized_keys"
    ]
  }
}
~
~
~
ami.pkr.hcl
```

packer init + packer build ami.pkr.hcl

```
==> amazon-ebs.amazon-linux:   Verifying        : runc-1.3.4-1.amzn2023.0.2.x86_64                    11/11
==> amazon-ebs.amazon-linux:
==> amazon-ebs.amazon-linux: Installed:
==> amazon-ebs.amazon-linux:   container-selinux-4:2.245.0-1.amzn2023.noarch
==> amazon-ebs.amazon-linux:   containerd-2.2.1-1.amzn2023.0.1.x86_64
==> amazon-ebs.amazon-linux:   docker-25.0.14-1.amzn2023.0.2.x86_64
==> amazon-ebs.amazon-linux:   iptables-libs-1.8.8-3.amzn2023.0.2.x86_64
==> amazon-ebs.amazon-linux:   iptables-nft-1.8.8-3.amzn2023.0.2.x86_64
==> amazon-ebs.amazon-linux:   libcgroup-3.0-1.amzn2023.0.1.x86_64
==> amazon-ebs.amazon-linux:   libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64
==> amazon-ebs.amazon-linux:   libnfnetlink-1.0.1-19.amzn2023.0.2.x86_64
==> amazon-ebs.amazon-linux:   libnftnl-1.2.2-2.amzn2023.0.2.x86_64
==> amazon-ebs.amazon-linux:   pigz-2.5-1.amzn2023.0.3.x86_64
==> amazon-ebs.amazon-linux:   runc-1.3.4-1.amzn2023.0.2.x86_64
==> amazon-ebs.amazon-linux:
==> amazon-ebs.amazon-linux: Complete!
==> amazon-ebs.amazon-linux: Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
==> amazon-ebs.amazon-linux: Provisioning with shell script: /var/folders/km/qcwvmr4x5j14x85706hdvb3w0000gn/T/packer-shell1211583436
==> amazon-ebs.amazon-linux: Stopping the source instance...
==> amazon-ebs.amazon-linux: Stopping instance
==> amazon-ebs.amazon-linux: Waiting for the instance to stop...
==> amazon-ebs.amazon-linux: Creating AMI custom-docker-ami-1774815821 from instance i-084375ca1d8360302
==> amazon-ebs.amazon-linux: Attaching run tags to AMI...
==> amazon-ebs.amazon-linux: AMI: ami-011c0a0eff69664d5
==> amazon-ebs.amazon-linux: Waiting for AMI to become ready...
==> amazon-ebs.amazon-linux: Skipping Enable AMI deprecation...
==> amazon-ebs.amazon-linux: Skipping Enable AMI deregistration protection...
==> amazon-ebs.amazon-linux: Terminating the source AWS instance...
==> amazon-ebs.amazon-linux: Cleaning up any extra volumes...
==> amazon-ebs.amazon-linux: No volumes to clean up, skipping
==> amazon-ebs.amazon-linux: Deleting temporary security group...
==> amazon-ebs.amazon-linux: Deleting temporary keypair...
Build 'amazon-ebs.amazon-linux' finished after 4 minutes 41 seconds.

==> Wait completed after 4 minutes 41 seconds

==> Builds finished. The artifacts of successful builds are:
--> amazon-ebs.amazon-linux: AMIs were created:
us-west-2: ami-011c0a0eff69664d5

yj@YJs-MacBook-Air packer %
```



setup AMI ID and public IP from aws console



terraform init

```
yj@YJs-MacBook-Air terraform % terraform init
Initializing the backend...
Initializing modules...
- vpc in modules/vpc
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.38.0...
- Installed hashicorp/aws v6.38.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

terraform plan

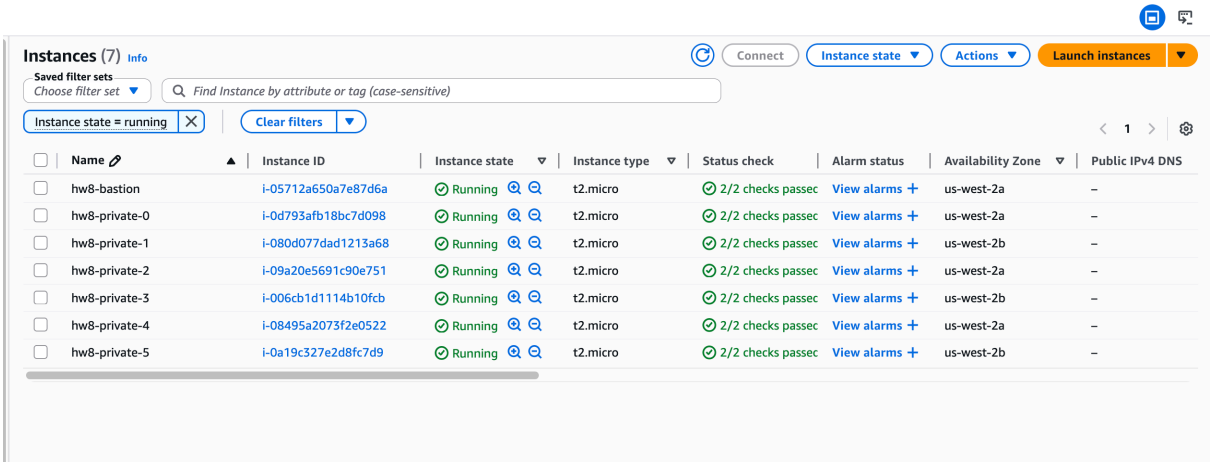
```
yj@YJs-MacBook-Air terraform % terraform plan
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 1s [id=ami-014d82945a82dfba3]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.bastion will be created
  + resource "aws_instance" "bastion" {
      + ami                                  = "ami-014d82945a82dfba3"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = true
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
```

terraform apply

```
yj@YJs-MacBook-Air terraform % terraform apply
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 1s [id=ami-014d82945a82dfba3]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicat
  + create

Terraform will perform the following actions:

  # aws_instance.bastion will be created
  + resource "aws_instance" "bastion" {
      + ami                                  = "ami-014d82945a82dfba3"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = true
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "hw8-deployer-key"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
```

```
module.vpc.aws_route_table_association.private[0]: Creation complete after 0s [id=rtbassoc-0b614b1bad0b17c30]
module.vpc.aws_route_table_association.private[1]: Creation complete after 0s [id=rtbassoc-0154735e05b23b4a2]

Apply complete! Resources: 24 added, 0 changed, 0 destroyed.

Outputs:

bastion_public_ip = "44.250.69.127"
private_instance_ips = [
  "10.0.3.242",
  "10.0.4.126",
  "10.0.3.132",
  "10.0.4.29",
  "10.0.3.177",
  "10.0.4.8",
]
yj@YJs-MacBook-Air terraform %
```

All instances launched



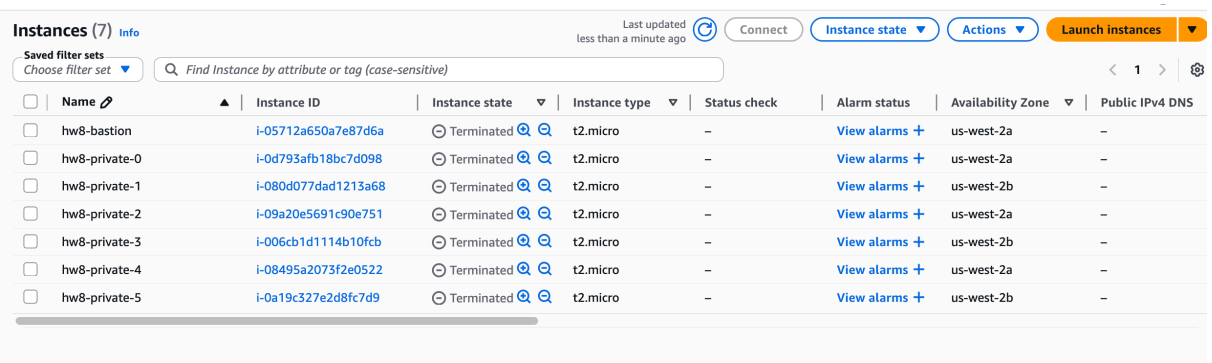
SSH tests public → private

```
yj@YJs-MacBook-Air terraform % ssh-add ~/.ssh/id_ed25519
Identity added: /Users/yj/.ssh/id_ed25519 (overdosedizzy@gmail.com)
yj@YJs-MacBook-Air terraform % ssh -A ec2-user@44.250.69.127
The authenticity of host '44.250.69.127 (44.250.69.127)' can't be established.
ED25519 key fingerprint is: SHA256:MZrv9LXBbZEO6ws2S3CQYoNEv7pW20Dw7/ni5C/Mj8Q
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '44.250.69.127' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-1-10 ~]$ ssh ec2-user@10.0.4.126
The authenticity of host '10.0.4.126 (10.0.4.126)' can't be established.
ED25519 key fingerprint is SHA256:uvmaog57D+NoZDT71lTJ3/KAmS8wSLINCtnn0+t/Si0.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.4.126' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-4-126 ~]$
```

```
       _/m/'
[ec2-user@ip-10-0-4-126 ~]$ docker --version
Docker version 25.0.14, build 0bab007
[ec2-user@ip-10-0-4-126 ~]$
```

Terraform destroy

```
module.vpc.aws_nat_gateway.nat: Still destroying... [id=nat-065acd7333b94be90, 00m50s elapsed]
module.vpc.aws_internet_gateway.igw: Still destroying... [id=igw-0552b9eae41f44559, 00m50s elapsed]
module.vpc.aws_internet_gateway.igw: Still destroying... [id=igw-0552b9eae41f44559, 01m00s elapsed]
module.vpc.aws_nat_gateway.nat: Still destroying... [id=nat-065acd7333b94be90, 01m00s elapsed]
module.vpc.aws_internet_gateway.igw: Destruction complete after 1m8s
module.vpc.aws_nat_gateway.nat: Still destroying... [id=nat-065acd7333b94be90, 01m10s elapsed]
module.vpc.aws_nat_gateway.nat: Destruction complete after 1m11s
module.vpc.aws_subnet.public[0]: Destroying... [id=subnet-00480b57f88bb4ec8]
module.vpc.aws_subnet.public[1]: Destroying... [id=subnet-04a7832118a33325e]
module.vpc.aws_eip.nat: Destroying... [id=eipalloc-00a1c4a461ec1ae3b]
module.vpc.aws_subnet.public[1]: Destruction complete after 0s
module.vpc.aws_subnet.public[0]: Destruction complete after 0s
module.vpc.aws_vpc.main: Destroying... [id=vpc-02740cf240ce2aaa4]
module.vpc.aws_eip.nat: Destruction complete after 1s
module.vpc.aws_vpc.main: Destruction complete after 1s

Destroy complete! Resources: 24 destroyed.
yj@YJs-MacBook-Air terraform %
```



Instances (7) Info

Last updated less than a minute ago | Connect | Instance state | Actions | Launch instances

Saved filter sets: Choose filter set

Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
hw8-bastion	i-05712a650a7e87d6a	Terminated	t2.micro	–	View alarms	us-west-2a	–
hw8-private-0	i-0d793afb18bc7d098	Terminated	t2.micro	–	View alarms	us-west-2a	–
hw8-private-1	i-080d077dad1213a68	Terminated	t2.micro	–	View alarms	us-west-2b	–
hw8-private-2	i-09a20e5691c90e751	Terminated	t2.micro	–	View alarms	us-west-2a	–
hw8-private-3	i-006cb1d1114b10fcb	Terminated	t2.micro	–	View alarms	us-west-2b	–
hw8-private-4	i-08495a2073f2e0522	Terminated	t2.micro	–	View alarms	us-west-2a	–
hw8-private-5	i-0a19c327e2d8fc7d9	Terminated	t2.micro	–	View alarms	us-west-2b	–

Summary

```
## Build the AMI:
cd packer
packer init .
packer build ami.pkr.hcl

## Deploy infrastructure:
cd terraform
terraform init
terraform plan
terraform apply

  ##  SSH to bastion:
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
ssh -A ec2-user@<BASTION_PUBLIC_IP>

## From bastion, hop to a private instance:
ssh ec2-user@<PRIVATE_INSTANCE_IP>

## Verify Docker on private instance:
docker --version

## Tear down:
cd terraform
terraform destroy
```

packer/ami.pkr.hcl

```
# aws ebs builder, launch temp EC2 instance from official A
mazon Linux AMI
  # plugin for amazon
  packer {
    required_plugins {
      amazon = {
        source  = "github.com/hashicorp/amazon"
        version = "~> 1"
      }
    }
  }

  # ssh public key to bake into the AMI, directly look into
local files
  variable "ssh_public_key" {
    type    = string
    default = "~/.ssh/id_ed25519.pub"
  }

  # base image and instance config
  source "amazon-ebs" "amazon-linux" {
    ami_name      = "custom-docker-ami-{{timestamp}}"
    instance_type = "t2.micro"
    region        = "us-west-2"
    ssh_username  = "ec2-user"

    # latest amazon linux 2023
    source_ami_filter {
      filters = {
        name                = "al2023-ami-*-x86_64"
        root-device-type    = "ebs"
        virtualization-type = "hvm"
      }
      most_recent = true
      owners      = ["amazon"]
    }
```

```
  }

  # install docker and inject ssh key
  build {
    sources = ["source.amazon-ebs.amazon-linux"]

    provisioner "shell" {
      inline = [
        "sudo dnf update -y",
        "sudo dnf install -y docker",
        "sudo systemctl enable docker",
        "sudo systemctl start docker",
        "sudo usermod -aG docker ec2-user"
      ]
    }

    provisioner "shell" {
      inline = [
        "echo '${file(pathexpand(var.ssh_public_key))}' >>
/home/ec2-user/.ssh/authorized_keys"
      ]
    }
  }
```

terraform/modules/vpc/main.tf

```
# vpc
  resource "aws_vpc" "main" {
    cidr_block = var.vpc_cidr

    tags = { Name = "hw8-vpc" }
  }

  # public subnets
  resource "aws_subnet" "public" {
    count             = length(var.public_subnet_cidrs)
    vpc_id            = aws_vpc.main.id
    cidr_block        = var.public_subnet_cidrs[count.inde
```

```
x]
    availability_zone = var.availability_zones[count.index]

    tags = { Name = "hw8-public-${count.index}" }
  }

  # private subnets
  resource "aws_subnet" "private" {
    count             = length(var.private_subnet_cidrs)
    vpc_id            = aws_vpc.main.id
    cidr_block        = var.private_subnet_cidrs[count.inde
x]
    availability_zone = var.availability_zones[count.index]

    tags = { Name = "hw8-private-${count.index}" }
  }

  # internet gateway for public subnets
  resource "aws_internet_gateway" "igw" {
    vpc_id = aws_vpc.main.id

    tags = { Name = "hw8-igw" }
  }

  # elastic ip for nat gateway
  resource "aws_eip" "nat" {
    domain = "vpc"

    tags = { Name = "hw8-nat-eip" }
  }

  # nat gateway in first public subnet, so private subnets
can reach internet
  resource "aws_nat_gateway" "nat" {
    allocation_id = aws_eip.nat.id
    subnet_id     = aws_subnet.public[0].id

    tags = { Name = "hw8-nat" }
```

```
  }

  # public route table: route internet traffic to igw
  resource "aws_route_table" "public" {
    vpc_id = aws_vpc.main.id

    route {
      cidr_block = "0.0.0.0/0"
      gateway_id = aws_internet_gateway.igw.id
    }

    tags = { Name = "hw8-public-rt" }
  }

  # associate public subnets with public route table
  resource "aws_route_table_association" "public" {
    count          = length(var.public_subnet_cidrs)
    subnet_id      = aws_subnet.public[count.index].id
    route_table_id = aws_route_table.public.id
  }

  # private route table: route internet traffic to nat gate
way
  resource "aws_route_table" "private" {
    vpc_id = aws_vpc.main.id

    route {
      cidr_block     = "0.0.0.0/0"
      nat_gateway_id = aws_nat_gateway.nat.id
    }

    tags = { Name = "hw8-private-rt" }
  }

  # associate private subnets with private route table
  resource "aws_route_table_association" "private" {
    count          = length(var.private_subnet_cidrs)
    subnet_id      = aws_subnet.private[count.index].id
```

```
    route_table_id = aws_route_table.private.id
  }

  terraform/modules/vpc/variables.tf:
  variable "vpc_cidr" {
    type = string
  }

  variable "public_subnet_cidrs" {
    type = list(string)
  }

  variable "private_subnet_cidrs" {
    type = list(string)
  }

  variable "availability_zones" {
    type = list(string)
  }

  terraform/modules/vpc/outputs.tf:
  output "vpc_id" {
    value = aws_vpc.main.id
  }

  output "public_subnet_ids" {
    value = aws_subnet.public[*].id
  }

  output "private_subnet_ids" {
    value = aws_subnet.private[*].id
  }
```

terraform/main.tf

```
provider "aws" {
    region = var.region
  }
```

```
  # vpc with public and private subnets
  module "vpc" {
    source = "./modules/vpc"

    vpc_cidr             = "10.0.0.0/16"
    public_subnet_cidrs  = ["10.0.1.0/24", "10.0.2.0/24"]
    private_subnet_cidrs = ["10.0.3.0/24", "10.0.4.0/24"]
    availability_zones   = ["${var.region}a", "${var.regio
n}b"]
  }

  # upload ssh public key to aws
  resource "aws_key_pair" "deployer" {
    key_name   = "hw8-deployer-key"
    public_key = file(pathexpand(var.ssh_public_key_path))
  }

  # bastion sg: only allow ssh from my ip
  resource "aws_security_group" "bastion" {
    name   = "hw8-bastion-sg"
    vpc_id = module.vpc.vpc_id

    ingress {
      from_port   = 22
      to_port     = 22
      protocol    = "tcp"
      cidr_blocks = [var.my_ip]
    }

    egress {
      from_port   = 0
      to_port     = 0
      protocol    = "-1"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
```

```
  # private sg: only allow ssh from bastion
  resource "aws_security_group" "private" {
    name   = "hw8-private-sg"
    vpc_id = module.vpc.vpc_id

    ingress {
      from_port       = 22
      to_port         = 22
      protocol        = "tcp"
      security_groups = [aws_security_group.bastion.id]
    }

    egress {
      from_port   = 0
      to_port     = 0
      protocol    = "-1"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }

  # latest amazon linux 2023 for bastion (no need for custo
m ami)
  data "aws_ami" "amazon_linux" {
    most_recent = true
    owners      = ["amazon"]

    filter {
      name   = "name"
      values = ["al2023-ami-*-x86_64"]
    }

    filter {
      name   = "virtualization-type"
      values = ["hvm"]
    }
  }

  # bastion host in public subnet
```

```
  resource "aws_instance" "bastion" {
    ami                         = data.aws_ami.amazon_linu
x.id
    instance_type               = "t2.micro"
    subnet_id                   = module.vpc.public_subnet_
ids[0]
    vpc_security_group_ids      = [aws_security_group.basti
on.id]
    key_name                    = aws_key_pair.deployer.key
_name
    associate_public_ip_address = true

    tags = { Name = "hw8-bastion" }
  }

  # 6 private ec2 instances using custom packer ami
  resource "aws_instance" "private" {
    count                  = 6
    ami                    = var.ami_id
    instance_type          = "t2.micro"
    subnet_id              = module.vpc.private_subnet_ids
[count.index % length(module.vpc.private_subnet_ids)]
    vpc_security_group_ids = [aws_security_group.private.i
d]
    key_name               = aws_key_pair.deployer.key_name

    tags = { Name = "hw8-private-${count.index}" }
  }
```

terraform/variables.tf

```
variable "region" {
    type = string
  }

  variable "my_ip" {
    type = string
  }
```

```
variable "ami_id" {
  type = string
}

variable "ssh_public_key_path" {
  type    = string
  default = "~/.ssh/id_ed25519.pub"
}

terraform/outputs.tf:
output "bastion_public_ip" {
  value = aws_instance.bastion.public_ip
}

output "private_instance_ips" {
  value = aws_instance.private[*].private_ip
}
```

terraform/output.tf

```
utput "bastion_public_ip" {
    value = aws_instance.bastion.public_ip
  }

  output "private_instance_ips" {
    value = aws_instance.private[*].private_ip
  }
```

terraform/terraform.tfvars

```
region = "us-west-2"
my_ip  = "MY.PUBLIC.IP/32"
ami_id = "ami-xxxxxxxxxxxxxxxxx"
```